# SQLite DB Tools

User Manual - version 1.0

**In short**

- SQLite databases, plain or encrypted, one in each window - open as many windows as you need
- Every table, form, search and query in its own tab; the Home tab shows everything you can do
- Tables, columns with rules, relations, indexes and automations kept by the database itself
- Import from CSV, Excel and OpenDocument; export to SQL, CSV, JSON; merge two databases
- Statistics, PDF prints, accounts with access levels, the operations log of every change
- Free of charge, no license key, in ten languages; up to 255 tables of 255 columns and 522,003 records per database

Copyright © 2026 Fation Coga. All rights reserved. Free of charge.

# Contents


1. **What SQLite DB Tools does**
2. **Installation and first start**
3. **The Home tab**
4. **Creating or opening a database**
   - New database
   - Opening a database
5. **Tables from files**
   - A new table
   - Required values the file does not have
   - More files into a table
   - Drag and drop, double click
6. **Designing a table**
   - Column types
   - Value of new records
7. **Adding entries**
8. **Browsing, searching and editing**
9. **Attachments**
10. **Connections between tables**
    - Relations
    - Indexes
    - Automations
11. **SQL queries**
12. **Export**
13. **Merging databases**
    - Preview before merging
    - Copying several tables
14. **Statistics**
    - Measures as in a spreadsheet
    - Over time
    - Formulas
15. **Printing documents**
16. **Accounts and access levels**
17. **The operations log**
18. **Security**
    - Protected databases
    - Lock
19. **Several databases at once**
20. **Preferences, theme, language**
21. **Help, About and troubleshooting**
22. **Keyboard shortcuts**
23. **License, trademarks and third-party software**
24. **The sample databases**
    - Calibration
    - Shop
    - Library

# 1 What SQLite DB Tools does

SQLite DB Tools creates, opens and works on **SQLite databases** - the single-file databases used by countless programs, phones and web sites. A database can be a plain SQLite file, readable by any SQLite tool, or **encrypted**: with an admin password and a recovery key, or with a passphrase in one of the formats of SQLite3 Multiple Ciphers (SQLCipher included).

In a database you design **tables** whose columns have types and rules (required, unique, smallest / largest, a pattern, a list of choices...), connect the tables with **relations**, speed them up with **indexes** and let the database keep itself in order with **automations**. Records are added with a form made for the keyboard, browsed and edited in a grid, searched, counted and summed in statistics, printed as PDF documents. Tables can come from **CSV, Excel and OpenDocument** files, and go out as **SQL** (SQLite, MySQL, PostgreSQL, SQL Server), **CSV**, **JSON** or a copy of the database. Accounts with five access levels decide who may do what, and the **operations log** keeps every change.

The program is **free of charge**: no license key, no registration. A database may have up to **255 tables**, each with up to **255 columns**, and up to **522,003 records** in all its tables together; a database beyond these limits opens read-only (chapter 4). It opens **one database per window**; to work on several databases, open the program several times (chapter 19).

> Everything you set in a database - column rules, relations, indexes, automations - is stored **in the database file itself**: it holds for every program that writes to that file, not only for SQLite DB Tools.

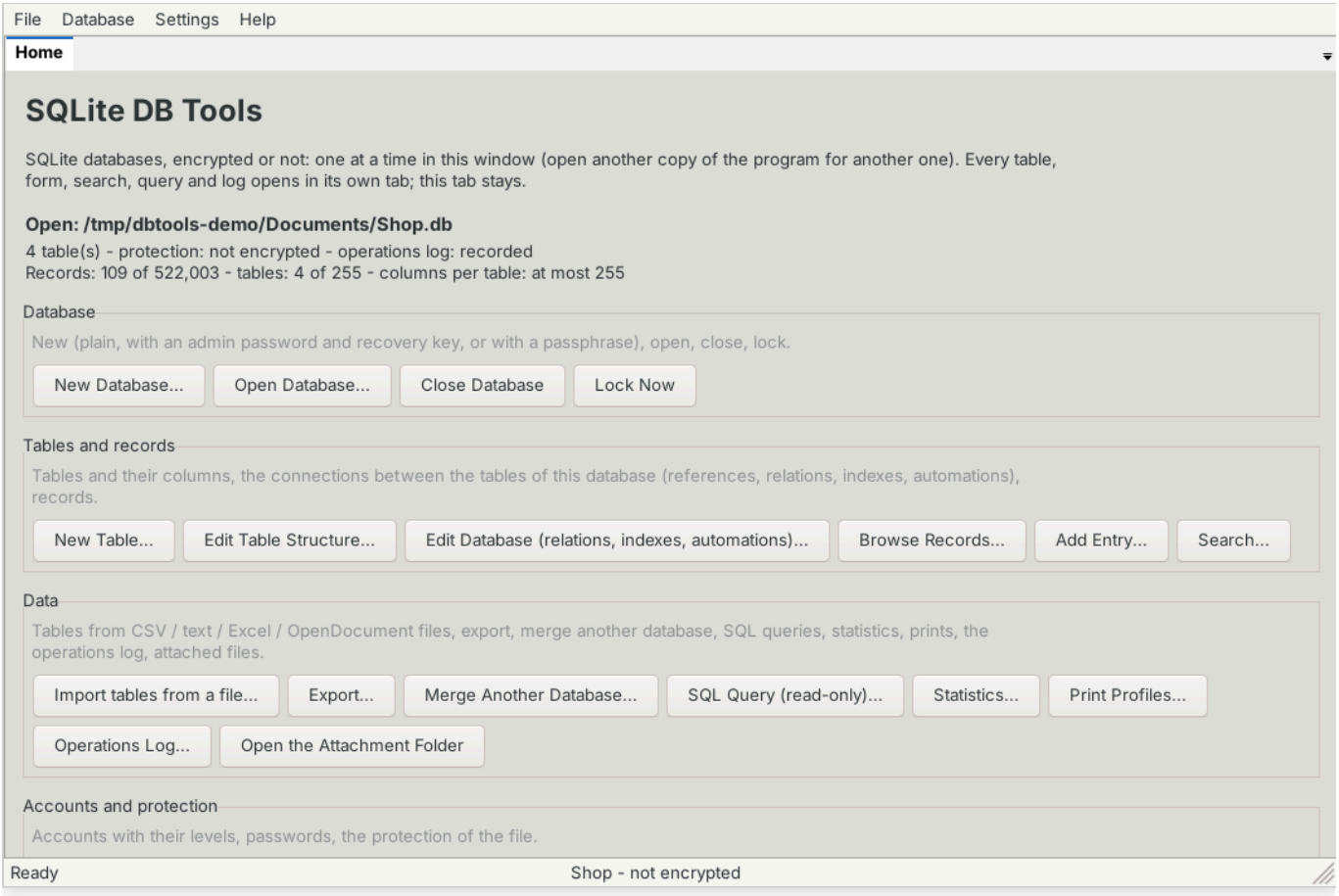
# 2 Installation and first start

Copy the program folder anywhere: it needs no installation (on a PC without Visual Studio, install the *Microsoft Visual C++ Redistributable x64* if Windows asks for it). Next to `SQLite DB Tools.exe` the program keeps the folder `SQLite DB Tools data` with its settings, its log and the language files; the user manuals are next to the exe. Your databases can be anywhere.

At start the program reopens the last database you used (its password, if any, is asked again: it is never stored) - **Settings > Application preferences** can switch this off. You can also start it with a database: `"SQLite DB Tools.exe" D:\Data\Shop.db`, a database dropped on the exe, or a double click on a database once the program is in Windows' "Open with" list (chapter 5).

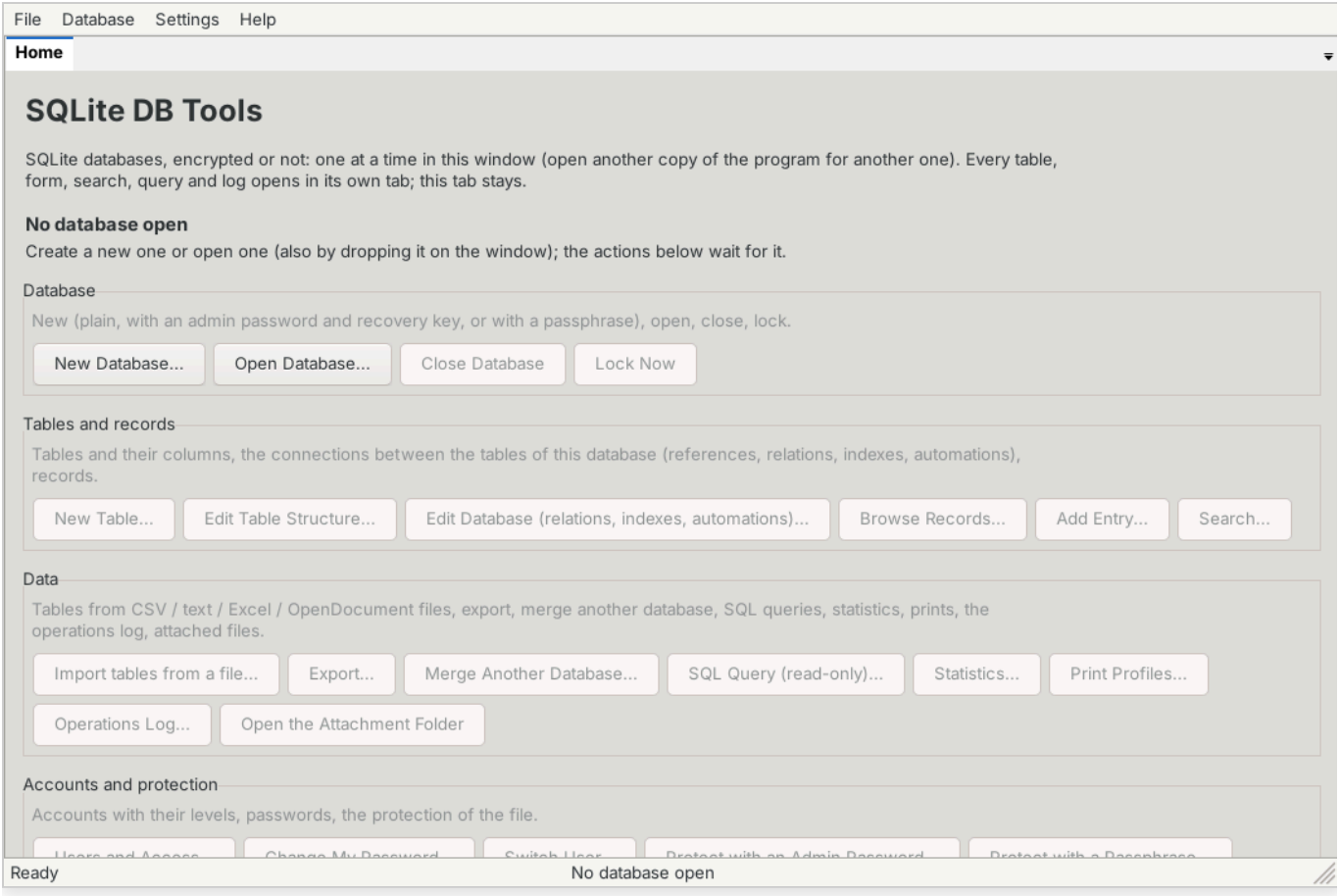
# 3 The Home tab

The first tab, **Home**, is always there and cannot be closed. It shows which database is open - its file, its number of tables, its protection, whether the operations log is recording, the account logged in and how much of the limits it uses (its records of 522,003, its tables of 255) - and **every action of the program as a button**, in four groups: *Database* (new, open, close, lock), *Tables and records* (new table, table structure, relations, browse, add entry, search), *Data* (import, export, merge, SQL, statistics, prints, operations log, attachments) and *Accounts and protection*. Below them, the **recent databases** open with one click.




Home with a database open: every action that can run is available.

A button is available only when its action can run: with no database open only **New Database**, **Open Database** and the recent databases are; the actions of an encrypted database appear only for an encrypted database; an account sees only what its level allows. Each button does exactly what the menu item of the same name does.



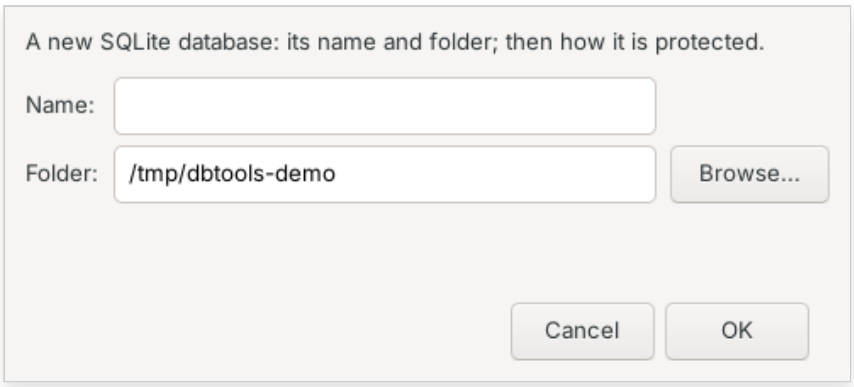

No database open: only creating or opening one is possible.

Every other window of work - a table's records, its structure, the entry form, a search, an SQL query, statistics, the relations, the operations log - opens in a **tab of its own** next to Home, so several tables can be open side by side. `Ctrl+W` closes the tab in front; a tab with changes not applied asks before closing.

# 4 Creating or opening a database

## New database

**File > New Database** (`Ctrl+N`) asks the **name** and the **folder** (Documents at first; `.db` is added) and says at once whether the name is free. A file that exists is never overwritten: you choose **Open the existing one**, **Create** it with a free name (*name (2)*) or **Change the name**. Then its protection:




File > New Database.

| Protection | What it means |
|---|---|
| Not encrypted | A normal SQLite file: anyone who has it can read it, with any SQLite tool. |
| Admin password + recovery key (recommended) | The file is encrypted with a random key. That key is kept in `<name>.db.keys` next to the database, locked twice: by your admin password and by a recovery key that is shown **once** (write it down). Either one opens the database. The password can be changed without re-encrypting anything. |
| A passphrase only | The file is encrypted directly with the passphrase, in the format you choose (SQLCipher 4 by default). The same passphrase opens the file in other tools such as DB Browser for SQLite. |

A new database has no tables yet: you are offered to make the first ones from a file (chapter 5), otherwise the structure editor opens (chapter 6).

## Opening a database

**File > Open Database** (`Ctrl+O`), **File > Open Recent**, or a recent database on Home. The program recognises the protection by itself: it asks the admin password when a `.keys` file is next to the database, a passphrase when the file is encrypted without one. In the passphrase window you can leave *Detect automatically*: every format of SQLite3 Multiple Ciphers is tried (SQLCipher 1-4, ChaCha20, AES-128/256, Ascon, AEGIS, RC4), so databases encrypted by other programs open with their passphrase too.

Databases made by other programs are welcome: their tables are shown, browsed, searched, edited and exported, and from the first opening the operations log records the work on them. Their structure can only grow by new columns.

A database beyond the limits of SQLite DB Tools - more than 255 tables, a table with more than 255 columns, or more than 522,003 records in all its tables together - opens **read-only**: it can be browsed, searched and exported, not changed. Each table shows its first 522,003 records and 255 columns, and the first 255 tables are listed. A message says which limit it is beyond, and Home says that it is read-only.

A database made by a version 1.x of SQLite DB Tools is **converted** to the format of version 2 when it is first opened (its tables of the program get their new names; the records do not change). The file as it was is kept next to it, named `<name>.format1.bak`: the versions 1.x open only that one. A file that cannot be written (read-only) is not converted: copy it where it can be written, then open it.

# 5 Tables from files

## A new table

**File > Import tables from a file...** (also offered when a database has no tables) reads **CSV and text files** (values separated by commas, semicolons, tabs or bars - found by itself, or chosen; text in quotes; whether the first row holds the names of the columns; UTF-8, UTF-16 or Windows encoding) and **spreadsheets**: Excel (`.xlsx`, and `.xls` of Excel 97-2003 and Excel 5 / 95) and OpenDocument (`.ods`, LibreOffice / OpenOffice). A workbook protected by a password is refused: save it without one first. A cell gives its value as the sheet shows it (a formula: its last result; a date: the date). The first rows are shown before anything happens.

A workbook with several sheets lists them with their columns and rows, in **groups** of sheets whose columns match (same names, any order). The checked sheets become **one table**: a column a sheet does not have stays empty for its rows. **Add a column with the name of the sheet** keeps where each row came from: sheets "Milano" and "Roma" with the same columns become one table with a column "Warehouse" holding Milano or Roma. **Make a table of each checked sheet** makes one table per sheet instead.

Then the columns of the new table: for each one the program guesses the **type** (text, whole number, decimal with point or comma, yes / no, date day/month/year or month/day/year, date and time), whether it is **required** (no empty value) and **unique** (every value different), with a few examples. Change what is wrong, untick the columns to leave out, name the table, **Import**. Nothing is written before every row is checked: rows that do not fit (a text in a number column, a repeated unique value...) are listed with their row and value - import the others, or go back and change the columns. The table and its records are made in one go, and the import is in the operations log. An import that would go beyond the 255 tables or the 522,003 records of the database writes nothing, and says which limit it reached.

The file is read again each time **Next** is pressed (or **Read the file again**): change it in Excel, save it, and go on without choosing it again.

## Required values the file does not have

When a required column is missing in the file, or some of its cells are empty, the program asks for them before checking the rows: **the same value everywhere**; **a value for each column**; **by hand**; or **leave those rows out**. By hand opens a grid with the rows that have empty required cells, 10 at a time: the empty cells are yellow and can be typed in, the others can be copied (`Ctrl+C`) and pasted into an empty one (`Ctrl+V`). `Tab` jumps to the next empty cell, along the row or, with the box ticked, down the column. **Confirm and import** becomes available when no required cell is left empty. With more than 150 empty cells the program warns that completing the original file is safer.

## More files into a table

The structure of a table (**Database > Edit Table Structure**) has **Import from file...**: one or more files add their rows to the table, each file shown and checked on its own. The file's columns are matched to the table's by name: it may have fewer columns (a column with a default value is filled by it; missing required values are asked as above); columns the table does not have stop the import, unless you

agree to **leave them out**. Every row is checked against the table's rules - also the unique values already in the table - before anything is written.

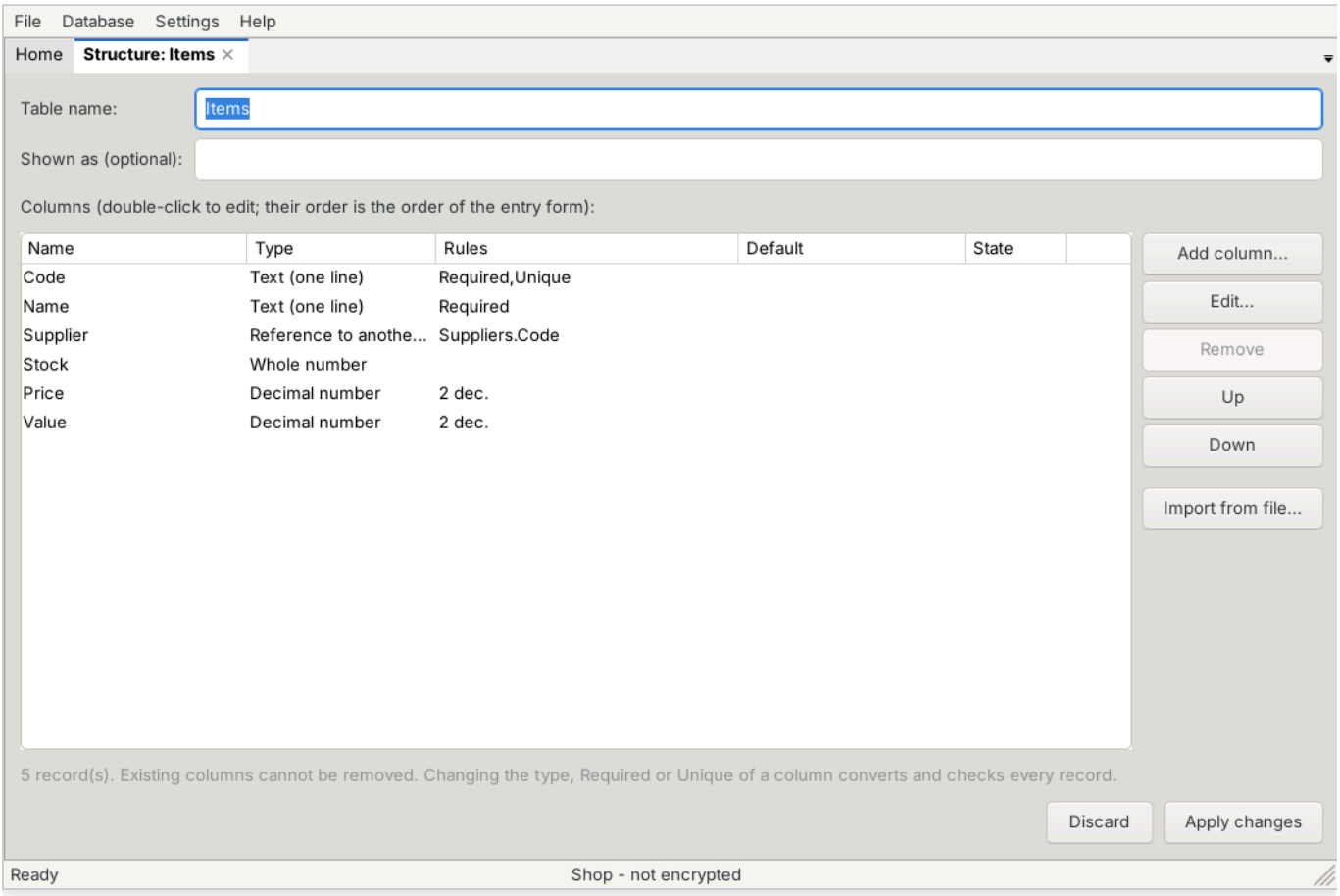
## Drag and drop, double click

Files dropped anywhere on the window go to the right tool: a database (`.db`, `.sqlite`..., encrypted or not, or its `.keys` file) opens - one per window, the others are named, to be opened in another window; CSV / text, `.xlsx`, `.xls` and `.ods` files are imported into the open database: as new tables or - when the tab in front is a table - their rows added to that table, as you choose. With no database open you are asked to create or open one first.

**Settings > Application preferences > Add SQLite DB Tools to 'Open with'** makes Windows offer the program for these files: choose once *Open with > Choose another app > SQLite DB Tools > Always* (Windows lets only you change a kind's default program). **Remove it** takes it away again.

# 6 Designing a table

**Database > New Table** or **Edit Table Structure** opens the structure editor in a tab. For a new table you can start from *How many details (columns) does an item have?*: the columns are created and the first one opens to be defined. Double-click a column to change it; **Up**/**Down** set the order of the entry form. A table can have up to 255 columns, a database up to 255 tables.




The structure of a table: name, type, rules and default of every column.

**Tables only grow.** Columns can be added, renamed and changed, but never removed. Changing the type, *Required* or *Unique* of a column converts and checks every record; if some record does not fit, nothing is changed and the reason is shown. A column that becomes required asks the value the existing records get.

## Column types

| Type | Holds | Optional rules |
|---|---|---|
| Text (one line) | names, codes | shortest / longest, pattern (regular expression) |
| Long text | notes on several lines | shortest / longest |
| Whole number | quantities | smallest / largest |
| Decimal number | prices, weights | smallest / largest, decimals (typing `3,5` or `3.5` both work) |
| Yes / No | flags | - |
| Date, Date and time, Time | dates | date format: YYYY-MM-DD, DD/MM/YYYY, MM/DD/YYYY, DD.MM.YYYY, DD-MM-YYYY (ISO is always accepted) |
| Choice from a list | one of fixed values | the choices |
| Attachment | a file | accepted files: images, PDF, spreadsheets, documents, archives, text, or any file |
| Reference | a value that must exist in a column of another table of the same database | the table and column (chapter 10) |

Every column can also be **Required** (never empty) and **Unique** (no two records with the same value), and can have a help text shown in the entry form.

## Value of new records

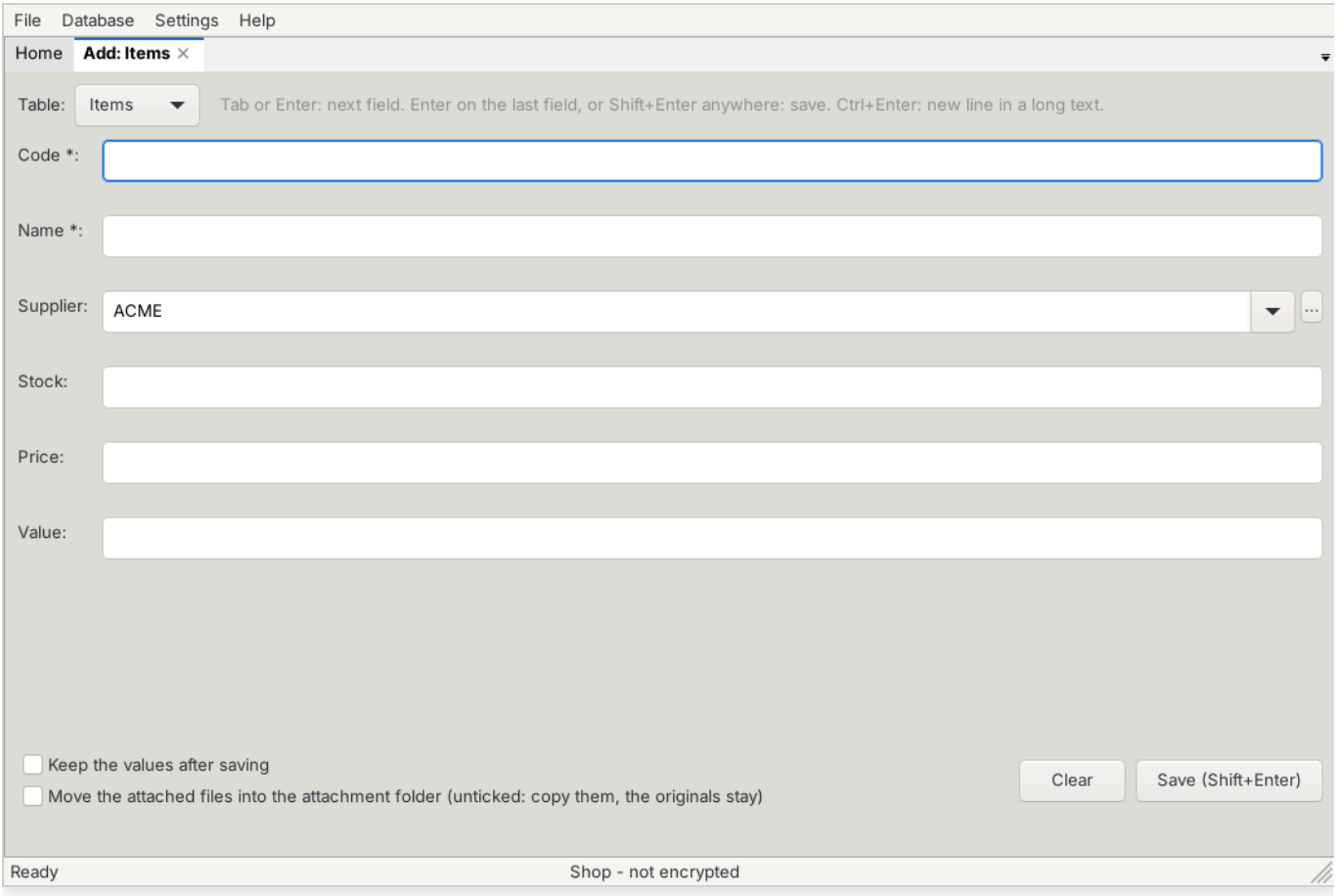
| Default | Example |
|---|---|
| Empty | - |
| A fixed value | `pcs` |
| Auto-increment number (start, step, zero padding) | `1001`, `00042` |
| Template | `{Name:3\|upper}-{seq:4}` gives `BOL-0001` for "Bolt M6" |
| Today / Now | the date / date and time of entry |
| Same as the last record | repeats the last value (e.g. the supplier) |
| Unique code (UUID) | `3f2a...-...` |
| Windows user name | who entered the record |

Template placeholders: `{Column}` the value of another column of the same record; `{Column:3}` its first 3 characters, `{Column:-3}` its last 3, `{Column|upper}` / `{Column|lower}` in capitals / small letters (they can be chained: `{Name:3|upper}`); `{seq}` a counter of this column, `{seq:4}` zero padded to 4 digits; `{date}`, `{date:YYYYMMDD}` (any layout of YYYY YY MM DD hh mm ss), `{time}`, `{user}`, `{table}`, `{db}`, `{uuid}`; `{{` and `}}` for braces. The column window shows an example while you type, and **Insert...** lists the placeholders and the other columns.

# 7 Adding entries

**Database > Add Entry** (`Ctrl+Shift+N`) opens the entry form of a table, made for the keyboard:

- `Tab` or `Enter`: next field. `Enter` on the last field saves.
- `Shift+Enter` in any field saves at once.
- `Ctrl+Enter`: a new line in a long text. `Space` ticks a Yes/No box.
- `Enter` in an empty attachment field opens the file chooser (showing only the accepted files).

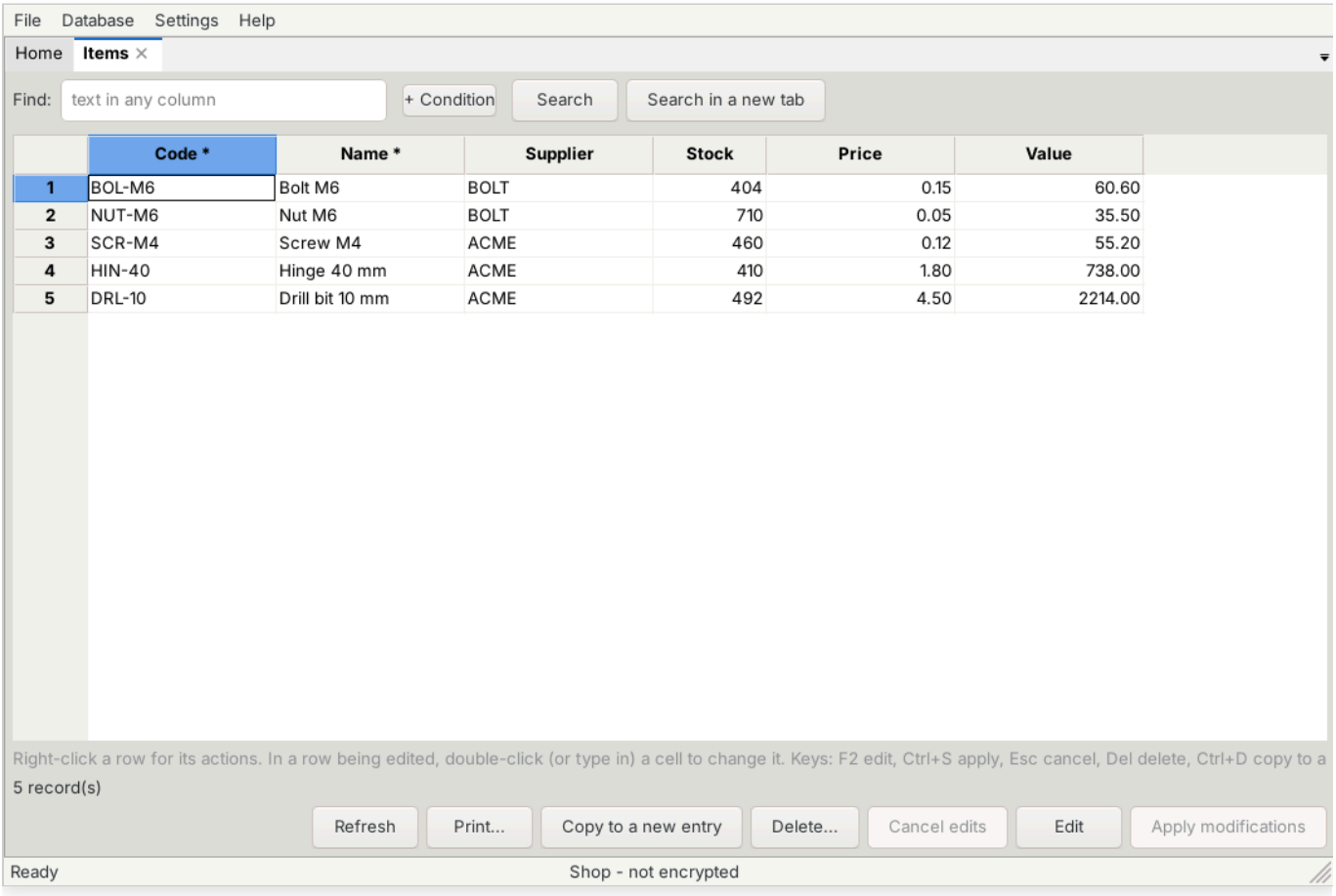



The entry form: a reference column offers the values of the other table.

Every field is checked while you type: the line under it turns red with the reason (for example *must be a whole number*). Before saving everything is checked again, including unique values against the records already in the database; on errors a list is shown and the first wrong field is selected. After saving the form is ready for the next record; tick *Keep the values after saving* to enter similar records quickly.

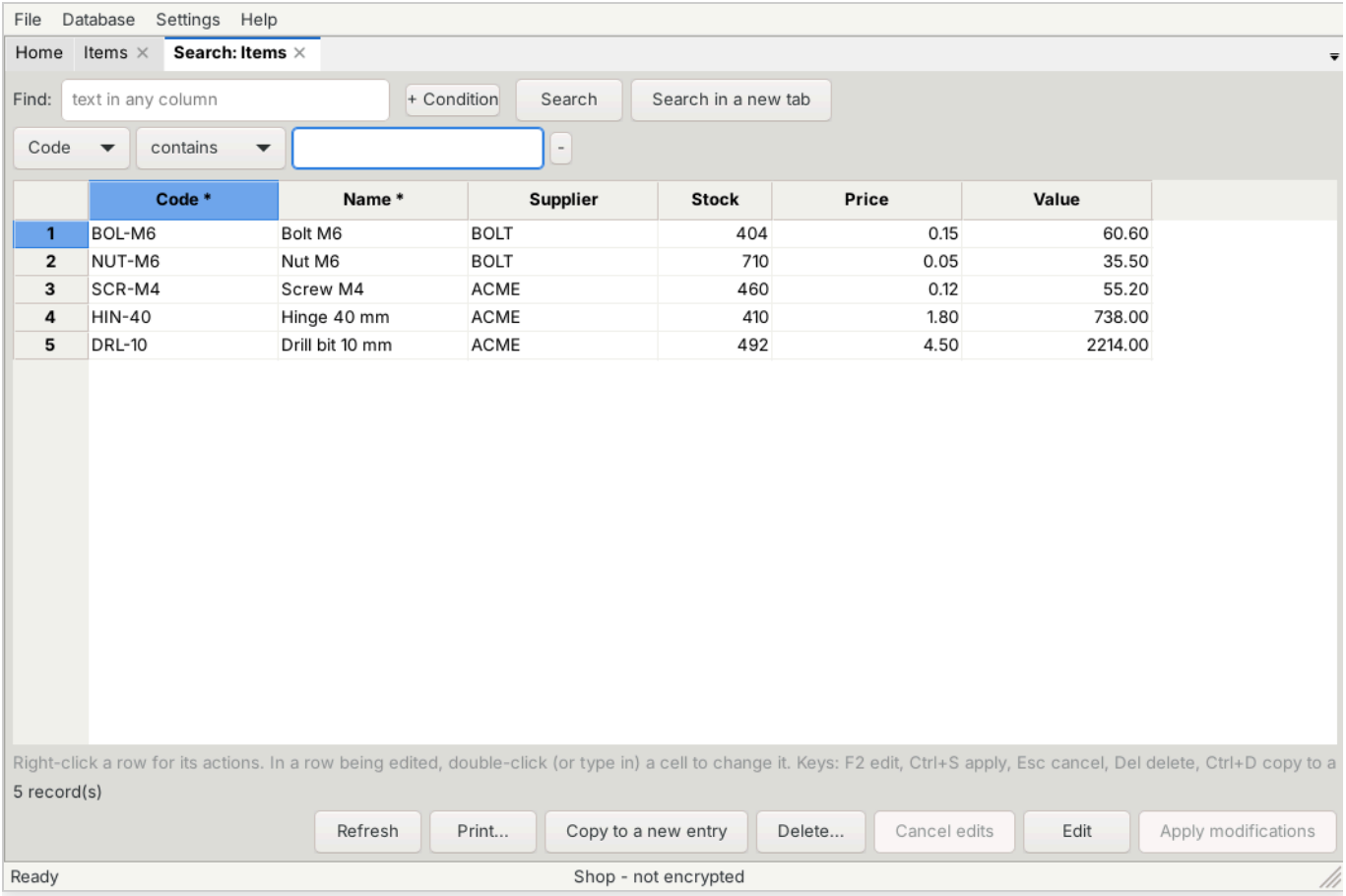
# 8 Browsing, searching and editing

**Database > Browse Records** (`Ctrl+B`) shows the records of a table in a grid. Click a column header to sort; large tables stay fast because the rows are loaded only when they are shown.




The records of a table.

**Find** searches a text in every column. **+ Condition** adds conditions *column* - *operator* - *value* (contains, is, is not, starts with, ends with, less / more than, at most / at least, is empty, is not empty) joined by *and* / *or*. **Search** filters the tab; **Search in a new tab** keeps the current one and shows the result beside it. **Database > Search** (`Ctrl+F`) opens an empty search.




A search with a condition.

Right-click a row and choose **Edit** (or press `F2`): the row turns yellow and its cells can be changed; changed cells turn orange. **Apply modifications** (`Ctrl+S`) applies every edited row together - unique values can even be exchanged between rows. Before applying, every edited record is checked: wrong cells turn red, the problems are listed and the first wrong cell is selected.

The other actions, in the right-click menu and as buttons: **Delete** (`Del`), **Copy to a new entry** (`Ctrl+D`), **Cancel edits** (`Esc`), **Refresh**, **Print...**, **History of this record** (every operation on it, from the operations log), and for attachments **Open**, **Show in its folder**, **Choose another file**, **Remove**.

# 9 Attachments

An attachment column holds a file. The file is copied (or moved, if *Move the attached files* is ticked in the form or in the preferences) into the folder `<database name>.files` next to the database, and renamed after its record: `Items_12_Photo.jpg` is the *Photo* of record 12 of *Items*. The database stores only this name, so it stays small and fast; keep the database and its folder together when you copy or back up. Files of deleted records and replaced files are moved to the `_deleted` subfolder, never deleted. **Database > Open the Attachment Folder** shows the folder.

**Where the folder is. Database > Attachment Folder Location** chooses it: relative to the folder of the database (the default, `<database name>.files`), relative to the folder of the program, or any folder. The database keeps the choice, so it travels with it. When the folder changes and files are already attached, you choose whether they are **moved**, **copied** or **left** where they are. A database that names a folder outside its own folder (received from someone, for example) asks once on each computer before that folder is used: until you confirm, the default folder is used. The Home tab shows the folder in use.

**The name of a new file.** In the definition of an attachment column, choose what the name of a new file is made of, in this order: the name of the table, the record number, the values of other columns of the record (as many as you like, in the order you set), and the name of the column itself, last before the extension (ticked by default). With the instrument's *Tag* and the *Date*, a *Calibration certificate* becomes `TH-042_2026-03-14_Calibration certificate.pdf`. The example under the choices shows the result. Files attached before keep their names.

**Missing files. Database > Check Attachments** looks for every attached file in the folder and lists, table by table, the records whose files are missing, with their values as in the records grid and the missing ones in red. Each missing file has its own **Fix** button: choose the file and it is put back into the folder under the name the record keeps (the record itself does not change, so a read-only database can be repaired too). **Check Again** runs the check again.

> Attachments are not encrypted, even when the database is.

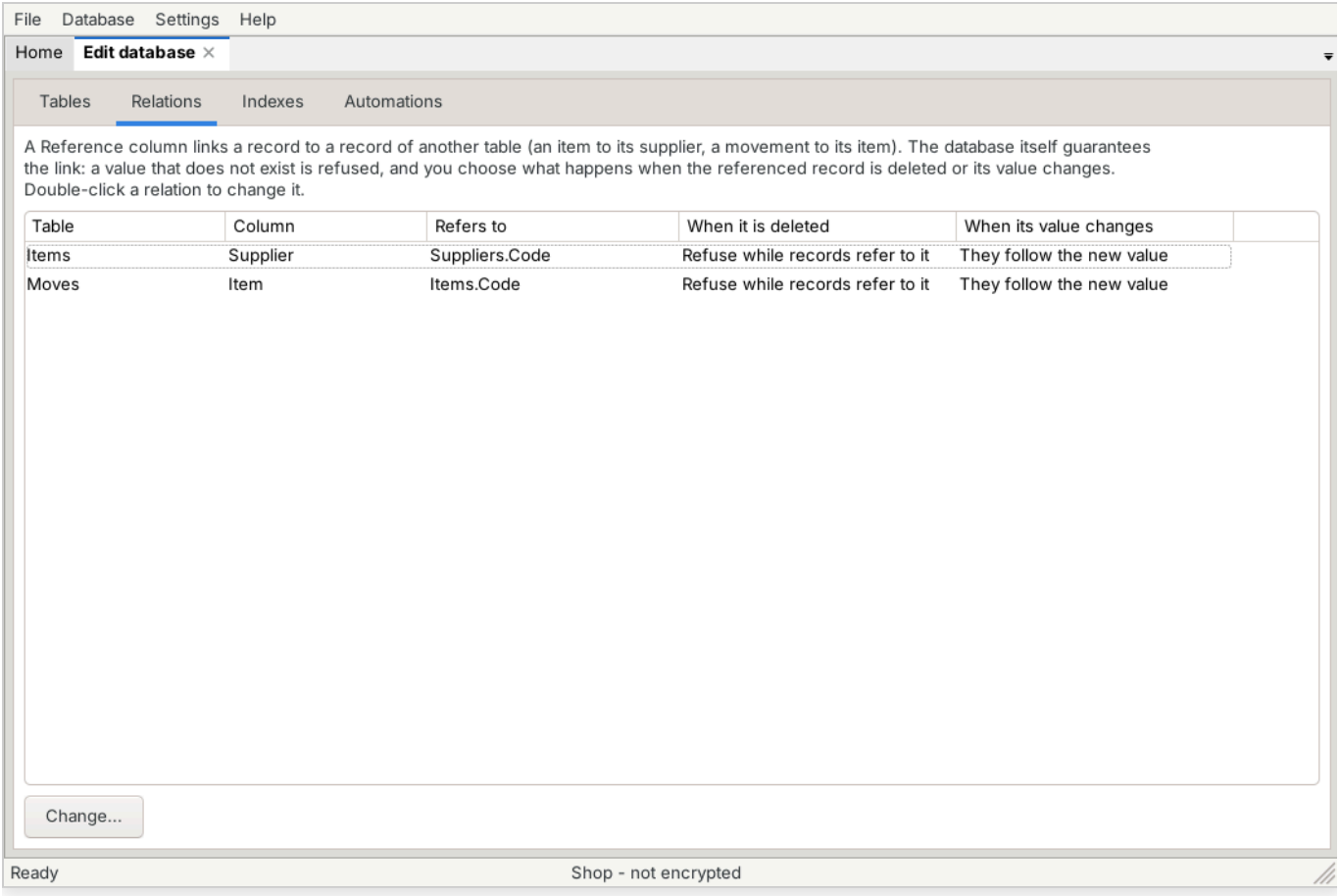
# 10 Connections between tables

**Database > Edit Database** (`Ctrl+Shift+D`) opens one tab with four pages: **Tables**, **Relations**, **Indexes** and **Automations**. What is set here is kept **by the database file itself** (as foreign keys, triggers and indexes), so it holds for every program that writes to it. It connects the tables **of the open database** - one database per window, so never with another database: to bring tables of another database in, merge them (chapter 13).

## Relations

A **Reference** column links a record to a record of another table: an item to its supplier, a movement to its item. The database refuses a value that does not exist, and you choose what happens to the records that refer to a record when it is **deleted** or its value **changes**:

| Choice | When the referenced record is deleted | When its value changes |
|---|---|---|
| Refuse while records refer to it | the deletion is refused (the safe default) | the change is refused |
| Delete them too / They follow the new value | the referring records are deleted as well | the referring records get the new value (default) |
| They lose the reference (empty) | the reference becomes empty | the reference becomes empty |




The relations of the sample database: items refer to suppliers, movements to items.

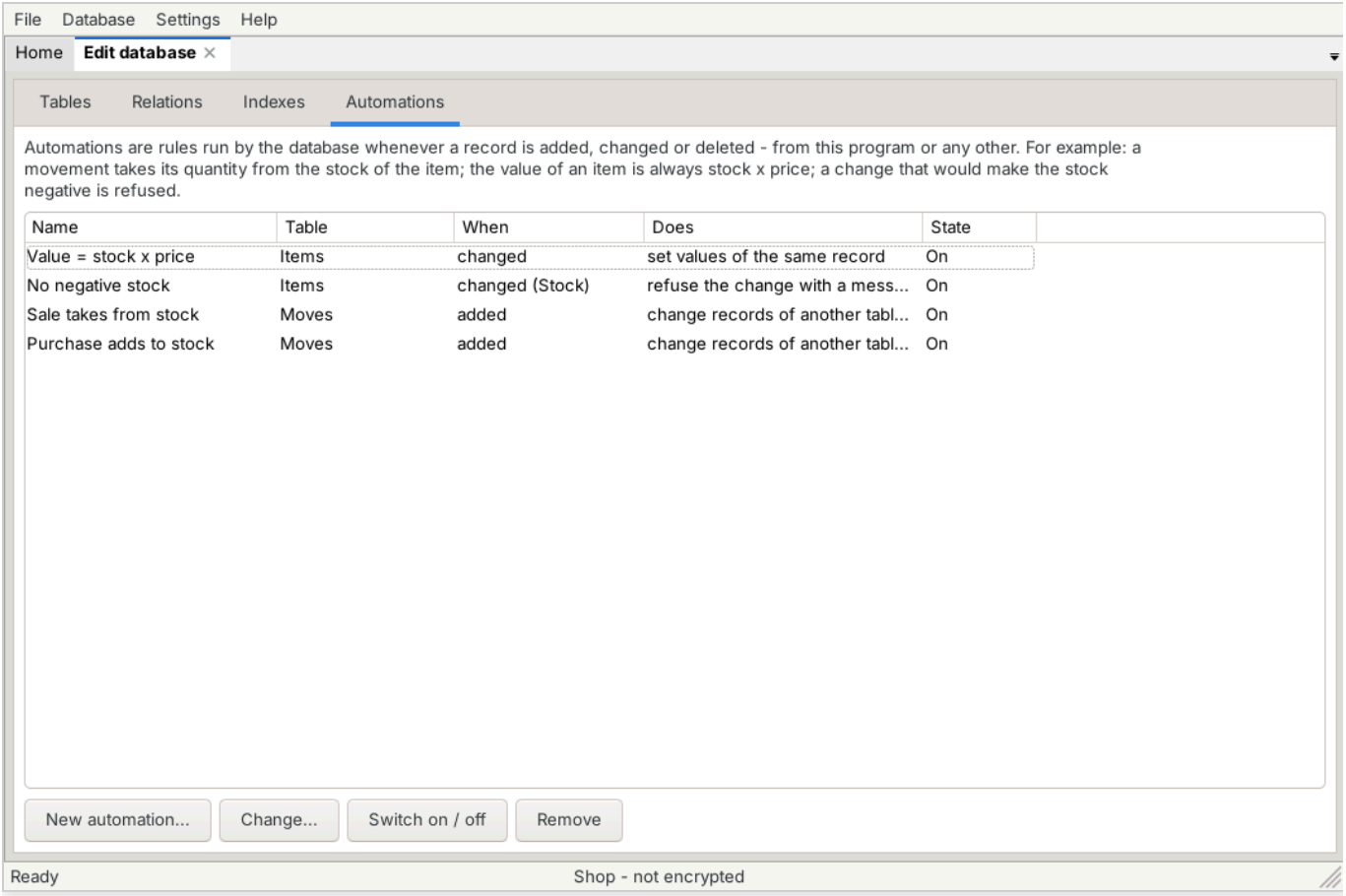
## Indexes

An index makes searching, sorting and the relations fast on large tables. Unique columns always have one; double-click any other column to add or remove its index. Index the reference columns and the dates you search by.

## Automations

An automation is a rule the database runs when a record of a table is **added**, **changed** (any column, or one column) or **deleted**, optionally **only if** a condition holds. It can **set values of the same record** (Value = round({Stock} * {Price}, 2)), **change records of another table** (with =, += or -=), **add a record to another table** (a line of a price history) or **refuse the change with a message** (the whole operation is cancelled).

Values are expressions: `{Column}` a value of the record, `{old.Column}` the value before a change, `{Reference>Column}` a value of the record it refers to, numbers, `'texts'`, `+ - * /`, `&` to join texts, and the functions `round(x, n)`, `abs`, `min`, `max`, `coalesce`, `upper`, `lower`, `length`, `trim`, `substr`, `today()`, `now()`, `date`, `year`, `month`, `day`, `days(from, to)`. **Check** tells at once whether the rule is correct. Automations can be switched off without removing them; removing one never changes data.




The automations of the sample database: the value of an item, the stock kept by the movements, no negative stock.

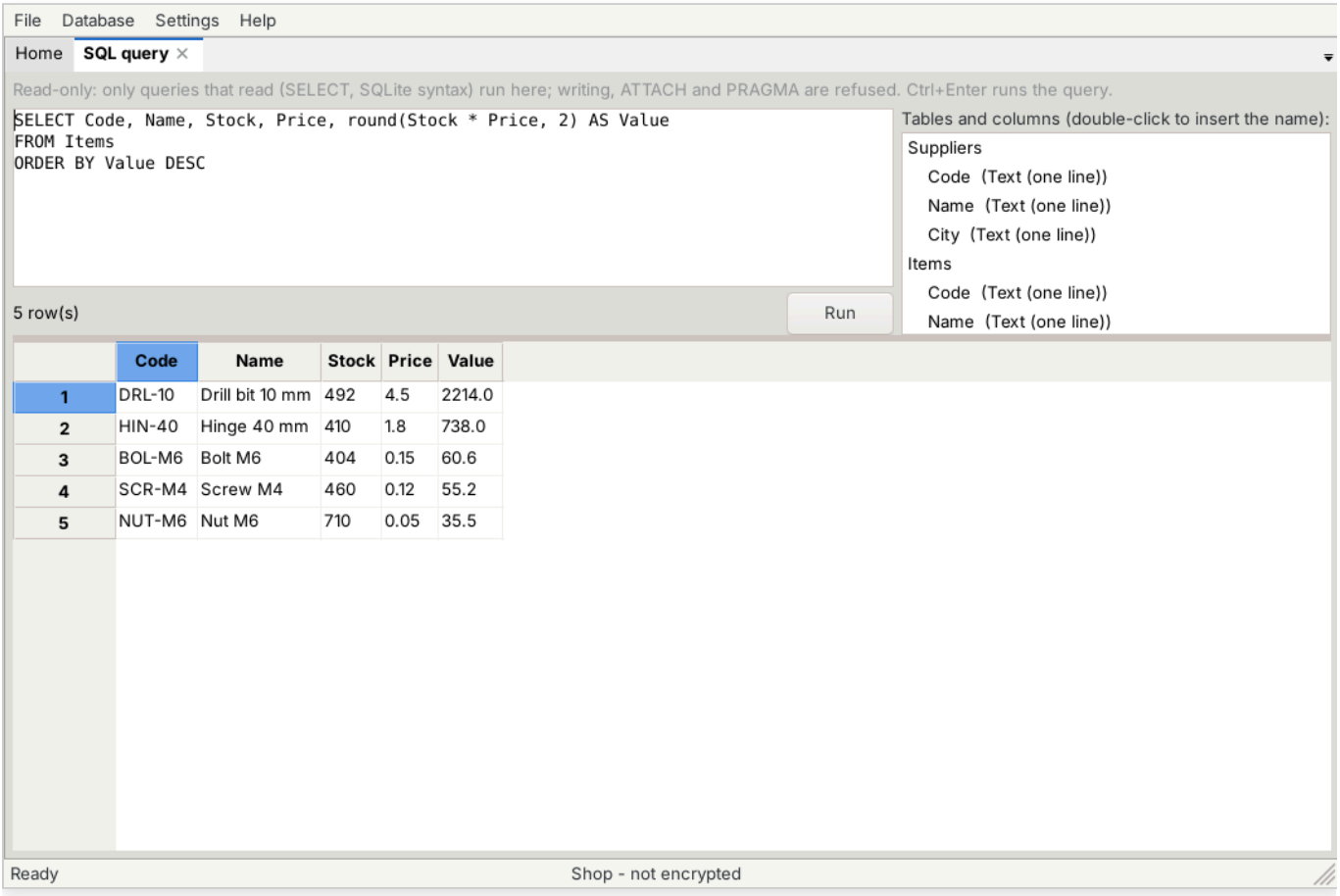
### Example - the stock follows the movements (the sample database)

1. When a record of **Moves** is *added*, **only if** `{Kind} = 'Purchase'`, **change records of another table**: **Items**, its records where **Code** = `{Item}`; set **Stock** += `{Qty}`.
2. The same **only if** `{Kind} = 'Sale'`, with **Stock** -= `{Qty}`.
3. When a record of **Items** is *changed* (column Stock), **only if** `{Stock} < 0`, **refuse the change** with "Not enough stock".

A sale of 5 pieces lowers the stock of that item by 5; a sale larger than the stock is refused, and the movement is not saved either.

# 11 SQL queries

**Database > SQL Query** (`Ctrl+Q`) runs one SELECT (SQLite syntax) and shows the result in a grid; `Ctrl+Enter` runs it. The tab is read-only: writing, `ATTACH` and `PRAGMA` are refused by SQLite itself, so a query can neither change the database nor reach other files. Double-click a table or column on the right to insert its name. At most 10,000 rows are shown at first; **Settings > Application preferences** raises it up to 1,000,000.



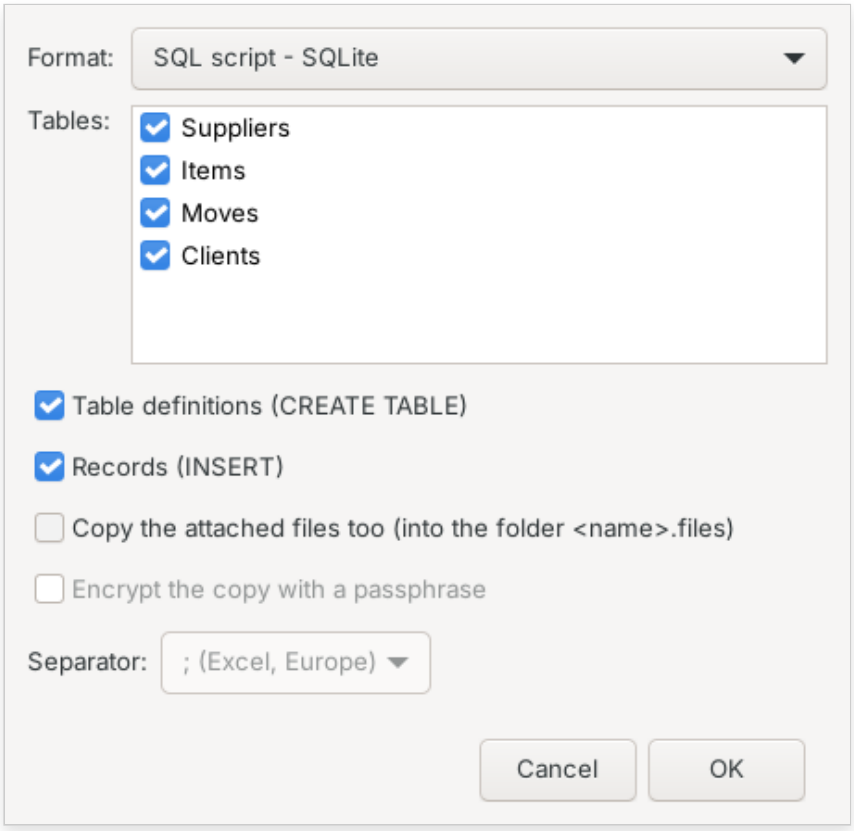

An SQL query with a calculated column.

# 12 Export

**File > Export** (`Ctrl+E`) writes the chosen tables as:

- an SQL script for **SQLite**, **MySQL / MariaDB**, **PostgreSQL** or **Microsoft SQL Server**, with the table definitions translated for that database and/or one INSERT per record;
- **CSV** for Excel (one file per table; cells that would start a formula are protected);
- **JSON**;
- a **copy of the database**, plain or encrypted with a new passphrase.

Tick *Copy the attached files too* to get them in `<export name>.files`.




File > Export.

# 13 Merging databases

**File > Merge Another Database** adds the records of a table of another database to a table of the open one (or to a new table). Choose the database (it is opened with its own password or passphrase), its table and the table here. If the two tables have the same columns, the records are merged as they are; otherwise a grid shows every column of the result: which column of the other database fills it, its name, what empty or missing values become and, for columns that exist only in the other database, whether to add them or drop them.

When an incoming record has a unique value that already exists, you are asked: **Skip it**, **Overwrite the record here** or **Keep both** (the value gets `_01`, `_02`... or the next free number); tick *Do the same for all the next conflicts* to answer once. Attachments are copied with new names. The records merged count towards the 522,003 of the database: when that limit is reached the merge stops there and says so.

## Preview before merging

**Preview...** shows every incoming record in a tab before anything is written. Edit any value; untick a row to leave it out (**Include All**, **Exclude All**, or a right click on the selected rows); a value shown in red cannot be saved as it is. For a record that is already here, its line shows both values where they differ: choose cell by cell (**Take the incoming value** or **Keep the existing value**, also for a whole column), and the row's **Action** - overwrite, keep both or skip. *Only rows with problems or already here* hides the others. **Apply** writes what is ticked; closing the tab writes nothing.

## Copying several tables

**Copy Several Tables...** copies tables of the other database as they are: their structure, records and attachments, with the relations and automations among them. Tick the tables to copy. When this database already has a table with the same name, you choose for each one:

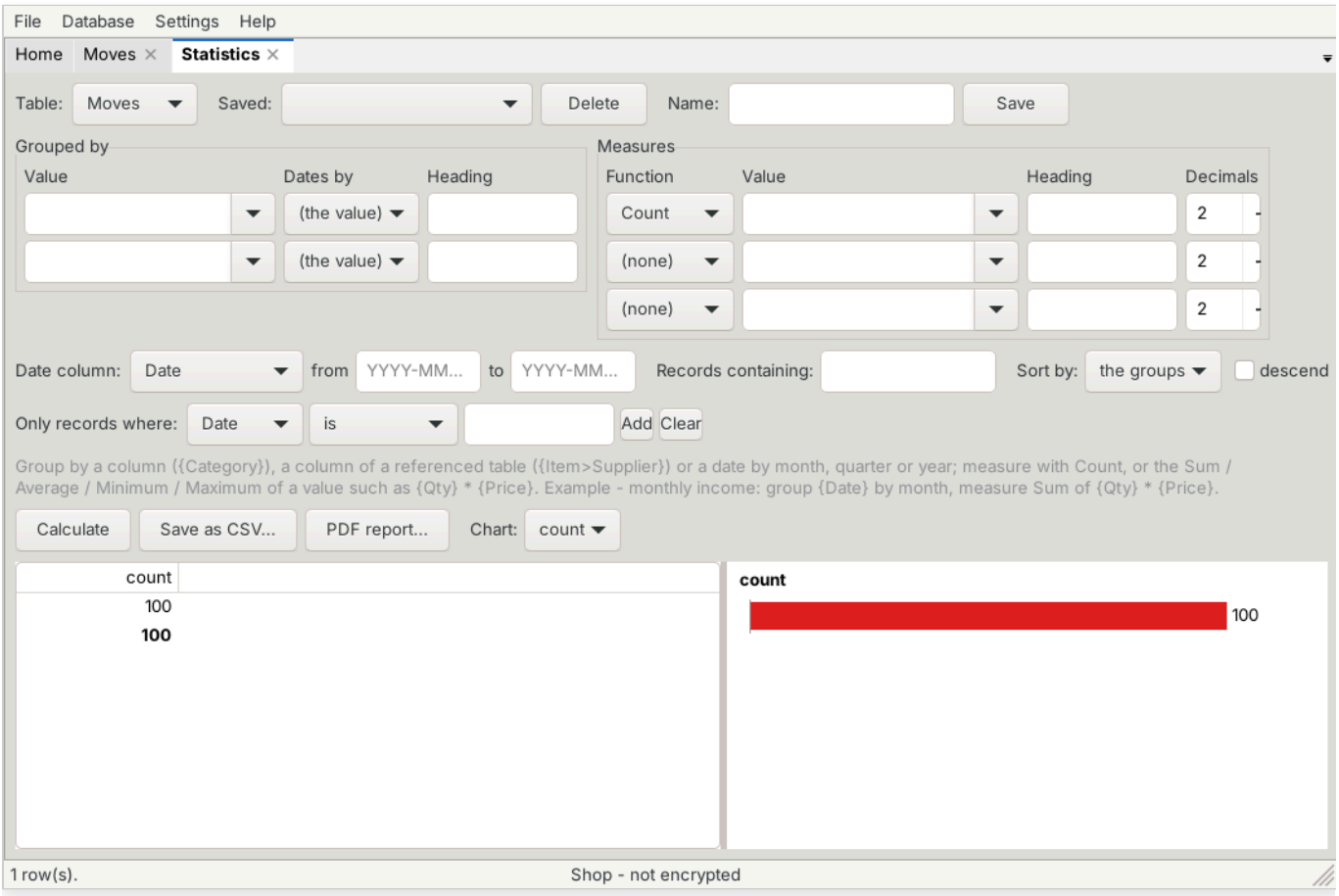
- **Give the copied table another name** (proposed: `Name_new`);
- **Rename the table of this database** (proposed: `Name_old`) and copy the other one under its name;
- **Replace the table of this database**: it is removed with its records, automations and print profiles. They are **lost permanently**: the warning says so in red, and a second confirmation is asked. A table still needed by others (references, automations) cannot be replaced: rename it instead.

# 14 Statistics

**Database > Statistics** (`Ctrl+Shift+T`) opens a statistics tab on the table of the current tab. A statistic groups the records and measures them:

- **Grouped by** up to two values: a column, a column of the referenced record (`{Item>Supplier}`), or a date by **day, week, month, quarter or year**. No grouping gives one line with the totals.
- Up to six **measures**, **M1** to **M6**: **Count** of the records, or the **Sum**, **Average**, **Minimum**, **Maximum**, **Count of different values**, **Median**, **Standard deviation** and **Variance** (of a sample or of all the values), **Product**, **First value** or **Last value** (by record number) of a value - a column or a formula such as `{Qty} * {Price}`; or a **Formula of the measures**.
- **Which records**: a date range, a text they contain, conditions *column - operator - value*.
- The **sort**: by the groups or by a measure.

**Calculate** shows the table, with a line of totals, and the chart of a measure (**Chart**, the first one at the start): grouped by a date, columns in time order, with a **Line** over them on request; grouped by anything else, bars with the largest first. With a second group its values are stacked in colours, with a legend. Every row is in the chart, which scrolls; the mouse on a column or a bar shows its exact values. **Save** keeps the statistic in the database under its name, for every user; **Save as CSV** writes the result for Excel; **PDF report** writes it as a PDF with the database, the period, the conditions, who made it and when.




A statistics tab on the movements.

## Measures as in a spreadsheet

Each measure has, besides its function and value:

- **Only when**: a condition on the record; only the records where it holds are measured - what SUMIF, COUNTIF and AVERAGEIF do in a spreadsheet. `{Kind} = "Sale"` sums only the sales; `{Qty} > 10 and {Customer} = "ACME"` combines two conditions.
- **Shown as**: **the value**, its **% of the total**, the **running total** (each row adds the rows above), the **rank** (1 = the largest), the **difference from the row before** or the **change (%) from the row before** (in the order of the rows: group by month, sorted by the groups, for the change month after month).
- A **Formula of the measures** calculates on the other measures of the same row, named `{M1}`, `{M2}`... or by their heading: `{M1} / {M2}` is the average price when M1 is the sum of `{Qty} * {Price}` and M2 the sum of `{Qty}`; `({Revenue} - {Cost}) / {Revenue} * 100` is the margin in %. It is calculated on the totals too.

Example - the money spent: table *Order items*, no grouping, M1 = **Sum** of `{Qty} * {Unit price}`. Grouped by `{Date}` by month, with M2 = **Sum** of the same value **shown as** running total, it gives the spending of each month and the amount so far.

## Over time

**Made of** chooses what the statistic counts. Besides **the records, grouped**, three choices follow something through time, period by period (**Every**: day, week, month, quarter or year), from the first date (or **from**) to today (or **to**). The first group, if any, splits each period into series (one per department, per item):

- **The records active in each period**: a record counts in every period between its **start** date and its **end** date (no end: still active). The staff of each month, the contracts running, the instruments in service.
- **The last record up to each period**: for each period, the last record dated up to its end, kept until a newer one. A price list with a "valid from" date gives the price of every month; readings give the last reading.
- **The past values of a column**: rebuilt from the operations log (chapter 17), which keeps every change with its time and the values before and after. A record never changed with the log on keeps its value of today; before its first change, the value it had then. Use the column in a measure (**Average** of `{Price}`) to see its history.

A count or a sum of the series is stacked in the chart; a price, an average or a minimum is one line for each series.

## Formulas

A value, a group or a condition is a formula of the columns of the record: `{Column}`, `{Reference>Column}` for a column of the referenced record, numbers, texts in quotes (`"ACME"`). **Functions...** lists them all:

| Kind | Operators and functions |
|---|---|
| Operators | `+ - * /`, `&` joins texts, `= <> < <= > >=` compare (texts without regard to case), `and`, `or`, `not` |
| Logic | `if(test, then, else)`, `isempty(x)`, `isnumber(x)`, `between(x, low, high)`, `contains(text, part)`, `startswith`, `endswith`, `coalesce(a, b)` |
| Numbers | `round(x, decimals)`, `roundup`, `rounddown`, `trunc`, `int`, `floor`, `ceiling`, `abs`, `sign`, `mod(a, b)`, `power(x, y)`, `sqrt`, `exp`, `ln`, `log10`, `log(x, base)`, `pi()`, `min(a, b)`, `max(a, b)` |
| Texts | `upper`, `lower`, `proper`, `trim`, `len`, `left(text, n)`, `right(text, n)`, `mid(text, start, n)`, `find(part, text)`, `replace(text, old, new)`, `concat(a, b)`, `rept(text, n)`, `text(x, decimals)`, `number(text)` |
| Dates | `today()`, `now()`, `year`, `month`, `day`, `hour`, `minute`, `weekday` (1 = Sunday), `weeknum`, `days(d1, d2)`, `months(d1, d2)`, `years(d1, d2)`, `adddays(d, n)`, `addmonths(d, n)`, `addyears(d, n)`, `eomonth(d, months)` |

The automations (chapter 10) use the same formulas, except `power`, `sqrt`, `exp`, `ln`, `log`, `log10`, `roundup`, `rounddown`, `trunc(x, decimals)` and `proper`: an automation is kept in the database and must work in any SQLite program, which does not have them.

# 15 Printing documents

Select records in a grid and choose **Print...** (button, right-click, or **Database > Print Selected Records**, `Ctrl+P`): they are printed as a **document** - an invoice, a delivery note, a pick list, a price list - saved as a PDF that then opens. How it looks is a **print profile** of the table; a table can have several, saved in the database for every user (**Database > Print Profiles** manages them without printing).

- **Document**: the title printed, the **number** (a template such as `INV-{date:YYYY}-{seq:5}`, whose counter moves on only when a document is really written), the layout (a table, or one block per record), the **issuer** and **notes**.
- **Columns**: which to print, in which order, with heading, alignment, width, bold; also the columns of the records they refer to (**Item > Name** prints the name of the item, not its code).
- **Amounts**: an amount per record (`{Qty} * {Price}`) added up at the end, with a tax, the currency and the decimals.
- **Client**: whether a client is asked when printing, and from which table.
- **Images**: a logo, a stamp, a signature, a **watermark** - each with its place, width, shape, opacity and whether it repeats on the following pages. The preview shows the page as it will be printed.

Every PDF of the program has a footer with the file name, the date and time it was written and the page (*Invoice 2026-0001.pdf | 2026-10-08 00:21 | Page 1 of 2*), so pages that got separated can be put back in order.

# 16 Accounts and access levels

Without accounts, whoever opens a database can do everything, and the operations log records the Windows user. With accounts, a **login** is asked every time the database is opened (and after it locked itself), and each account can do only what its **level** allows:

| Level | May |
|---|---|
| **Master** | manage the accounts only (create, change level, lock, new password) - no work on the data |
| **Full access** | everything: delete records, tables and relations, automations, security, merge; manages the accounts of the three levels below |
| **Modify data** | read, add and change records (not delete); export, statistics, print, SQL queries |
| **Entry only** | add new records - nothing else, not even read the others |
| **Read only** | browse, search, export, statistics, print |

The menus and buttons an account may not use are greyed out; the database refuses the operation anyway, also if it is tried another way.

**Database > Users and Access** on a database without accounts offers to switch them on: you choose the name and password of a **Master** account and of a first **Full access** account (at least 6 characters each), then you log in. From then on nobody can open the database without an account. The window lists every account with its level and state: **New account**, **Change level**, **Lock** / **Unlock**, **New password**. Nobody can change or lock their own account.

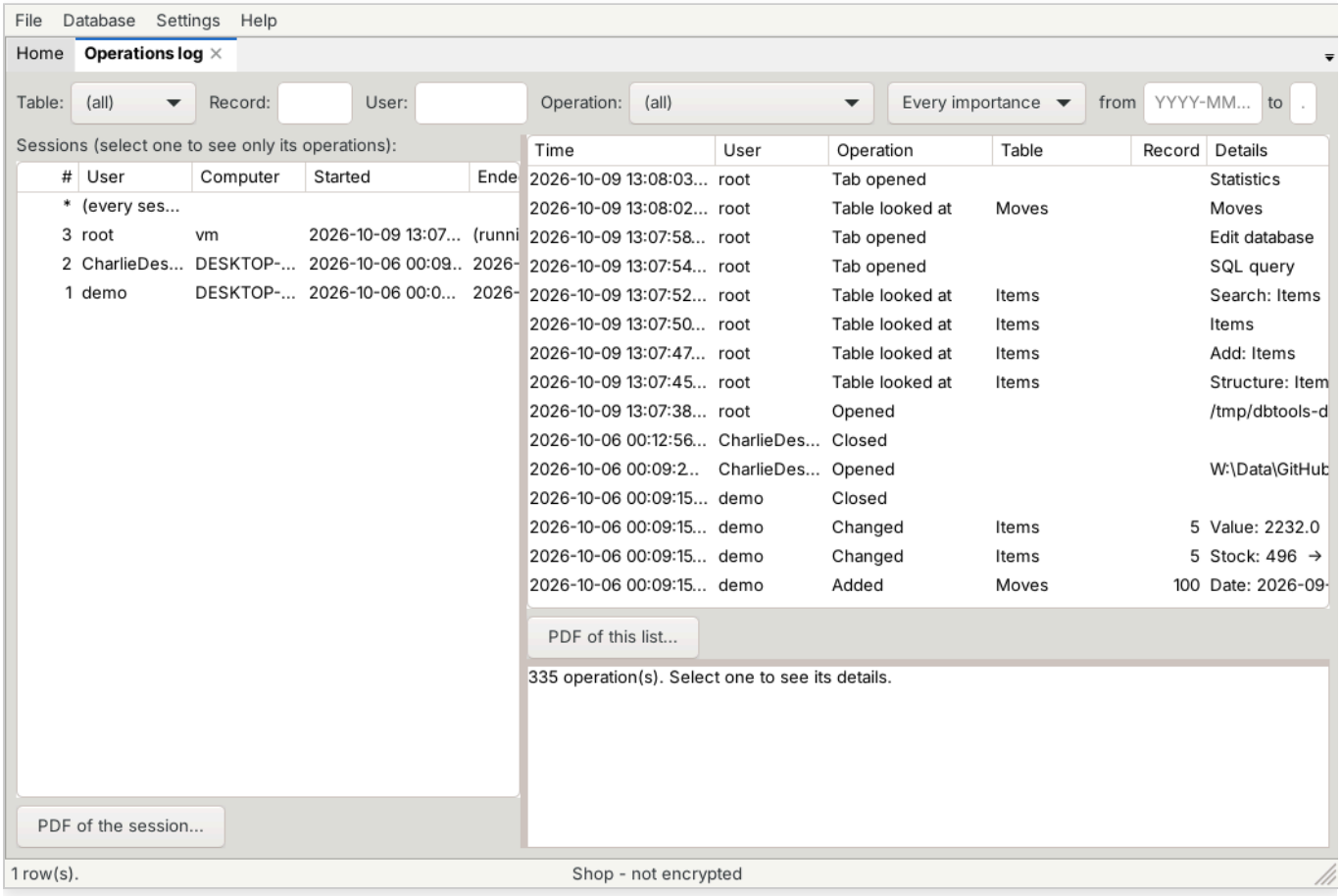
> **Accounts are never deleted.** When someone leaves, lock their account: their name stays in the operations log next to everything they did. Keep the Master password safe - nobody can recover it.

**Database > Change My Password** changes the password of the account logged in; **Database > Switch User** closes the tabs and asks a new login, for a shared PC.

# 17 The operations log

Next to every database the program keeps its operations log: `operations_<name>.db` (for `Shop.db`, `operations_Shop.db`), encrypted with the same key as the database. It records the **sessions** - who opened the database, on which PC, from when to when - and in each session every **operation**: records added, changed (with the values before and after) and deleted, tables, relations and automations changed, logins (also the refused ones), accounts managed, exports, statistics, prints, and the tabs opened and searches made. The changes of records are written by the database itself, so nothing escapes.

**Database > Operations Log** (`Ctrl+Shift+H`) shows the sessions on the left and the operations on the right, newest first; filter them by table, record, user, kind, importance and dates. Selecting an operation shows its details: for a change, every value that changed, *old → new*. Each line has an **importance** from 1 (routine) to 5 (critical), the **time of the device** and the same moment in **UTC**, and the **device** (computer and Windows user). **PDF of the session** and **PDF of this list** write the operations as a PDF report.




The operations log: sessions on the left, their operations on the right.

> The operations log is read-only for everybody: no account can change or delete it. Copy and back it up with the database.

# 18 Security

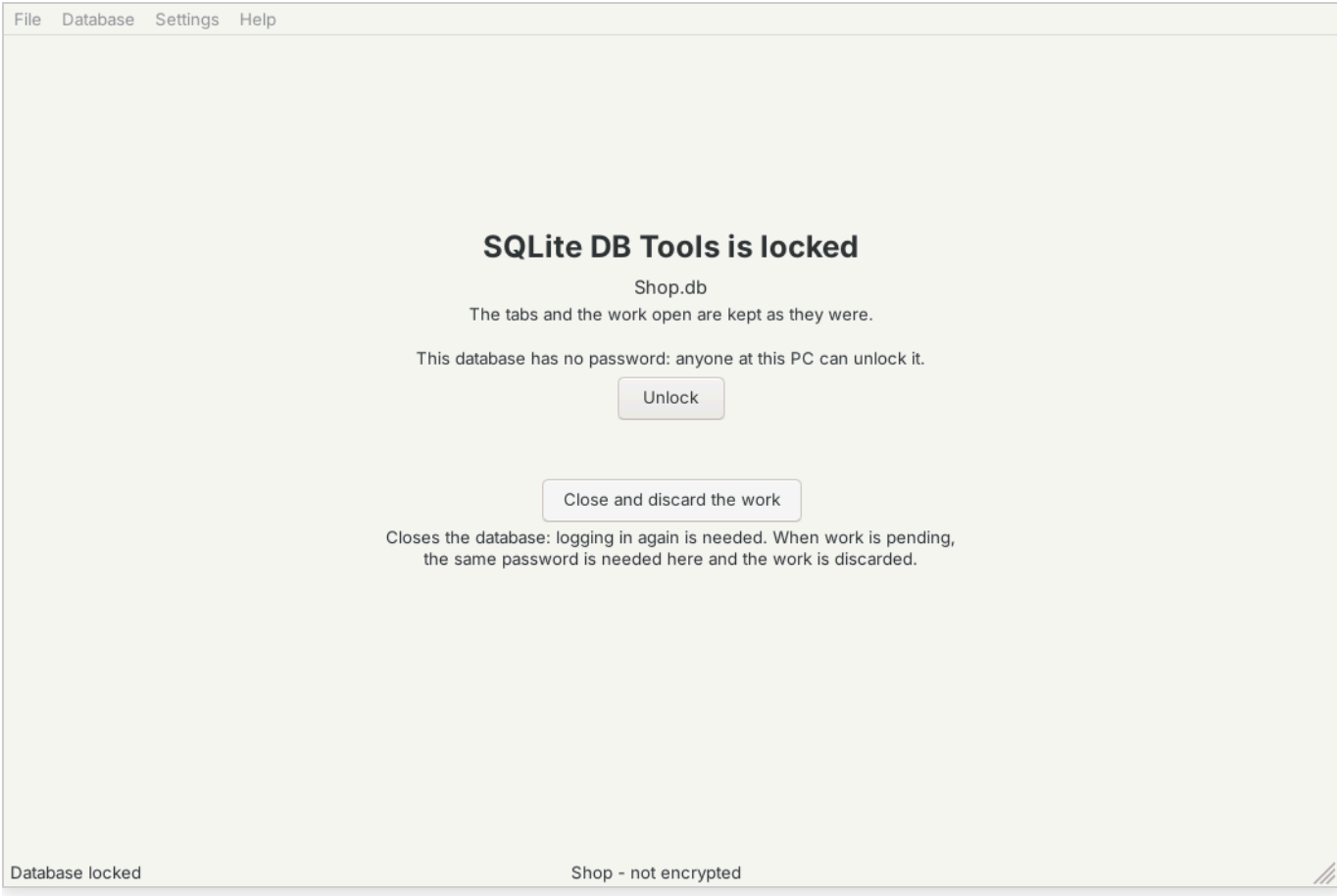
## Protected databases

**Database > Security** holds, for an admin-password database: **Change the Admin Password**, **New Recovery Key**, **Rotate the Encryption Key** (re-encrypts everything with a new key, if a copy with its key file or the password may have leaked), **Database Format** (SQLCipher 4, ChaCha20, AES-256) and **Show the Database Key** (to open a copy with another tool). For a passphrase database: **Change the Passphrase** and **Remove the Encryption**. Any plain database can be **protected** later with an admin password or a passphrase.

> Back up the database, its `.keys` file, its `.files` folder and its operations log together, and keep the recovery key away from the PC. Without the password or recovery key (or the passphrase) nobody can open an encrypted database - not even the author of the program.

## Lock

**File > Lock Now** (`Ctrl+L`) covers the window at once; it also locks by itself after a time without activity - **10 minutes** by default, set in **Settings > Application preferences** (0 = never). A bar warns 30 seconds before. The lock **keeps the work**: the tabs and the changes not applied stay as they were, under the lock screen, until the password of the account (or the admin password, or the passphrase) is typed. After the third wrong password each attempt waits longer. **Close and discard the work** closes the database instead.




The lock screen: the work stays under it.

Values typed by you are never pasted into SQL: they are always passed separately to SQLite. Names of tables and columns are used only after they were matched with the database's own structure.

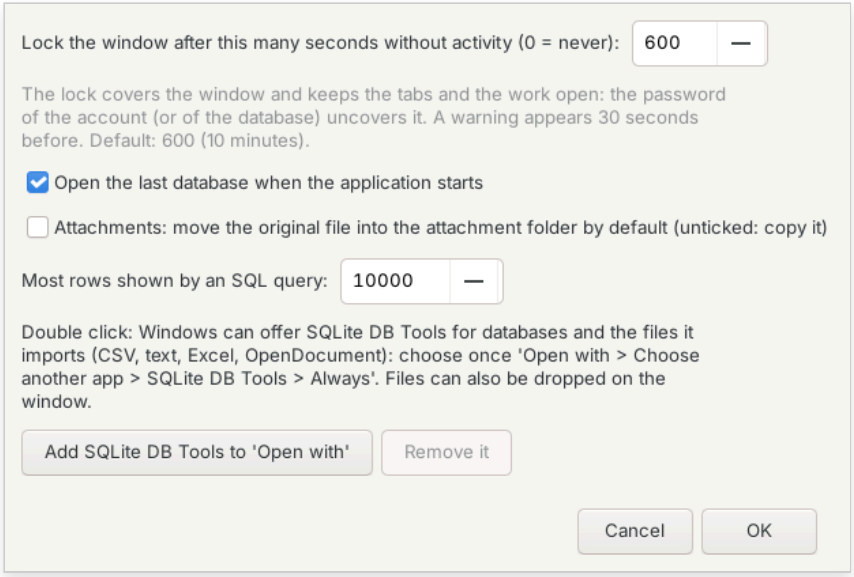
# 19 Several databases at once

Each window works on **one database**: its tabs, its relations, its lock and its operations log belong to that database only. To work on two databases at the same time, **start the program again** (from the Start menu, the taskbar or a double click on the other database): every copy is a window of its own, with its own database, and they run side by side without disturbing each other. Dropping a second database on a window opens it in that window after closing the first one - the files it cannot take are named, to be opened in another window.

The same database can be open in two windows too: SQLite takes turns between them, and each window sees the other's changes after **Refresh**. For changes of structure (tables, relations, automations) keep the database open in one window only.

# 20 Preferences, theme, language

**Settings > Application preferences**: the time before the lock, whether the last database opens at start, whether attachments are moved or copied, the most rows of an SQL query, and the "Open with" entries of Windows.

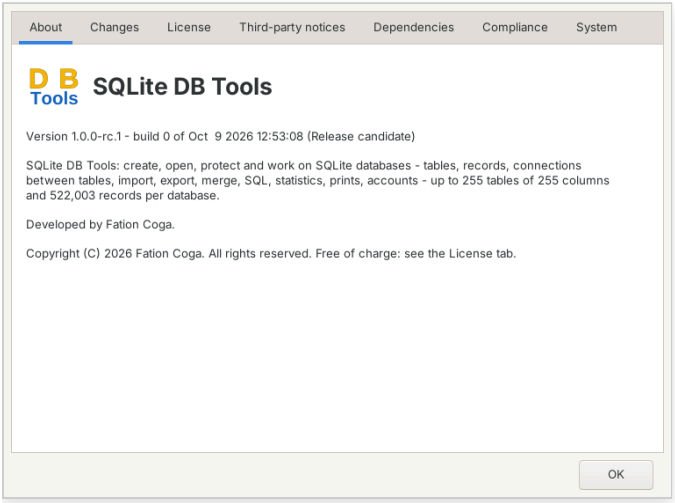



Settings > Application preferences.

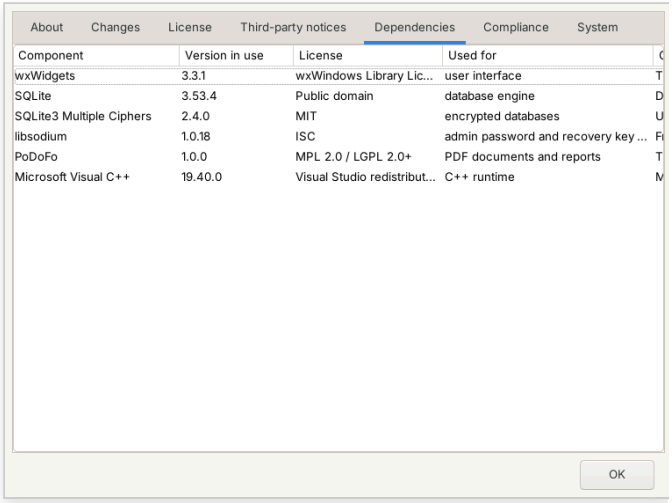
**Settings** also sets the **Theme** (light / dark), the **Language** - English, Italian, French, German, Russian, Spanish, Portuguese, Chinese, Hindi, Japanese or Arabic (right to left) - and the **Log Level**, and opens the data folder. Theme and language can be used at once or at the next start. The user manual opens in the language of the program (F1).

# 21 Help, About and troubleshooting

**Help > About** has one tab per topic: About (version and developer), Changes, License, Third-party notices, Dependencies (the components and their versions), Compliance and System (copy it when you ask for help). **Help > Application Log** shows what the program did; the full log is `SQLite DB Tools data\log.txt`.




Help > About.




The components the program is built on.

| Problem | What to do |
|---|---|
| "The key file is missing" | Put the `.keys` file back next to the database (from your backup). |
| The passphrase is refused | Check the passphrase; choose the format explicitly instead of *Detect automatically*. |
| "The file ... is not in the attachment folder" | The `.files` folder was not copied with the database. |
| "Your account is not allowed to do this" | The level of the account does not allow it (chapter 16). |
| "Records of ... still refer to it" | A relation protects the record (chapter 10): change or delete the records that refer to it first. |
| "Refused by an automation: ..." | An automation of the database refused the change; the message says why. |
| A dropped file is not opened | One database per window: open the others in another window (chapter 19). |
| "The database holds at most 522,003 records..." | The database is full: delete the records you no longer need, or move some tables into another database (Export as a copy, then delete them here). |
| Home says "Read-only" | The database is beyond the limits (chapter 4): export what you need, or merge it table by table into a database within them. |

# 22 Keyboard shortcuts

| Keys | Action |
|---|---|
| `Ctrl+N` / `Ctrl+O` | New / open database |
| `Ctrl+L` | Lock now |
| `Ctrl+E` | Export |
| `Ctrl+W` | Close the tab |
| `Ctrl+Shift+N` | Add entry |
| `Ctrl+B` / `Ctrl+F` | Browse / search |
| `Ctrl+Q` | SQL query |
| `Ctrl+Shift+D` | Edit database (relations, indexes, automations) |
| `Ctrl+Shift+T` | Statistics |
| `Ctrl+Shift+H` | Operations log |
| `Ctrl+P` | Print the selected records |
| `F1` | This manual |
| `Alt+X` | Exit |
| Entry form: `Tab`, `Enter`, `Shift+Enter`, `Ctrl+Enter` | next field, next field or save on the last, save, new line |
| Grid: `F2`, `Ctrl+S`, `Esc`, `Del`, `Ctrl+D` | edit row, apply, cancel row, delete, copy to a new entry |

# 23 License, trademarks and third-party software

SQLite DB Tools is written and owned by Fation Coga and given **free of charge**: anyone may use it, at home or at work, on any number of computers, with no license key, and pass on unmodified copies free of charge. It may not be sold, modified or reverse engineered; its source code is not published. It comes without warranty. The full text is `LICENSE.md`, next to the program and in **Help > About > License**.

The author of SQLite, D. Richard Hipp, gave his code to the public domain with a blessing in place of a legal notice. This program does not share its source, so it does not claim that blessing; it gives what it can - itself, free of charge - and asks only a smaller wish in return:

> *May your backups be recent and your restores never needed.*
> *May every query return what you meant to ask.*
> *May you keep your data in order, and may it keep you out of trouble.*

The program is built on software of other authors, used under the licences of their owners, who keep every right on it:

- **wxWidgets** - the wxWidgets team and contributors, wxWindows Library Licence: www.wxwidgets.org
- **SQLite** - D. Richard Hipp and the SQLite developers, public domain; SQLite is a registered trademark of Hipp, Wyrick & Company, Inc.; SQLite DB Tools is not made, endorsed or supported by them: www.sqlite.org
- **SQLite3 Multiple Ciphers** - Ulrich Telle, MIT licence: utelle.github.io/SQLite3MultipleCiphers
- **SQLCipher** - a trademark of Zetetic LLC; the format is read through SQLite3 Multiple Ciphers: www.zetetic.net/sqlcipher
- **libsodium** - Frank Denis, ISC licence: libsodium.org
- **PoDoFo** - the PoDoFo developers, MPL 2.0 / LGPL: github.com/podofo/podofo
- **DB Browser for SQLite** - the DB Browser for SQLite project: sqlitebrowser.org
- **Microsoft**, **Windows**, **Excel**, **Visual Studio** and **Segoe UI** - trademarks of Microsoft Corporation; **LibreOffice** - The Document Foundation; **OpenOffice** - the Apache Software Foundation
- **PDF** - the Portable Document Format, ISO 32000; **MySQL** - Oracle Corporation; **PostgreSQL** - the PostgreSQL Global Development Group; **MariaDB** - MariaDB Foundation

All other names of products and companies are trademarks of their respective owners. Their use only names them: it implies no endorsement by, or affiliation with, them. The names in the sample database are invented.

# 24 The sample databases

Three sample databases come with the program, to try everything on real-looking data without building a database first. **Help > Sample Databases** (or the button on the Home tab) opens one: you work on a **copy** in the folder `Samples` of the data folder, so the original stays as it is. Choosing the same sample again asks whether to open your copy with your changes or to start again from a fresh copy. The data is invented, with today set to 1 October 2026: dates "overdue" or "due soon" are counted from there.

| Sample | Tables | What it shows best |
|---|---|---|
| Calibration | Locations, Suppliers, Instruments, Calibrations, Movements | references, automations that keep the instruments up to date, attachment names, missing attachments |
| Shop | Categories, Suppliers, Items, Customers, Movements | stock kept by the movements, statistics over a year of sales, a rule that refuses a change |
| Library | Authors, Books, Members, Loans | a record that changes another one twice (lent, back), a loan refused, overdue searches |

## Calibration

The measuring instruments of a company: where each one is, when it was calibrated and when it is due again. An instrument has a *Tag* (CA-001), a type, a location and a supplier (references), a photo and its manual (attachments). Every calibration refers to its instrument and keeps its certificate as a PDF named after the instrument and the date.

1. **Browse** *Instruments*: sort by *Next due*; search *Next due* **is less than** 2026-10-01 to find the instruments overdue.
2. **Add an entry** to *Calibrations* for one of them, with a result *Pass* and a *Valid until* date: the automation sets *Last calibrated*, *Next due* and *Status* of the instrument. With *Fail* the instrument goes *Out of service*.
3. Attach a certificate to the new calibration: its file is named `<Tag>_<Date>_Calibration certificate.pdf`. **Edit Table Structure** of *Calibrations*, column *Calibration certificate*, shows how the name is chosen.
4. **Database > Check Attachments**: three certificates are missing. Fix one with any PDF file.
5. **Add an entry** to *Movements* moving an instrument to *VAN-1*: its *Location* follows. Moving *OS-008* (retired) is refused with a message.
6. Try to delete a location still used by an instrument: the relation refuses it. **Statistics** on *Calibrations*: their number and cost per laboratory.

## Shop

A small shop with 120 items in five categories, their suppliers, 45 customers and a year of stock movements. The *Stock* of an item is never typed: the movements change it (purchases and returns add, sales take away).

1. **Browse** *Items* and note the stock of an item; **add an entry** to *Movements*: a *Sale* of 2 of it. Its stock is 2 lower; the operations log shows both changes as one operation.

2. Change the *Price* of an item to less than its *Cost*: the automation refuses it.
3. **SQL**: the items to order again, `SELECT SKU, Name, Stock FROM Items WHERE Stock < "Reorder level"`; or **Search** *Items* where *Stock* **is less than** 5.
4. **Statistics** on *Movements*: sales per month, quantity per category (through the reference to the item).
5. The photos are named after the *SKU* and the *Name* of the item, without the table and the column: see the column *Photo* of *Items*. Some items have no photo.
6. **Export** the customers to CSV, or **print** a list of the movements of one item.

## Library

Classic books, their authors, the members of the library and two years of loans. A book is *Available*, *Lent* or *Lost*; the loans change it.

1. **Add an entry** to *Loans* for an available book: the book becomes *Lent*. Lending it again is refused: "This book is not available".
2. Edit the loan and fill *Returned on*: the book is *Available* again (an automation on a change of one column).
3. **Search** *Loans* where *Returned on* **is empty** and *Due* **is less than** 2026-10-01: the loans overdue.
4. **History of this record** on a book: every loan that changed it.
5. **Add an entry** to *Members*: the *Card* number is proposed (auto-increment from 1001).
6. Try **Database > Attachment Folder Location** on your copy: move the covers into another folder and see the Home tab name the new folder.